October completion plan

Four members: A, B, C and D | Start: 13 October 2026

Finish by 30 October. Reserve 31 October for essential repairs.

Complete by	Task IDs	Required outcome
14 Oct 2026	T01-T04	Scope, environment, data/IR checks and shared tracker; every member traces the pipeline.
15 Oct 2026	T05-T08	Joint approval of scoring, preprocessing, evidence handling and evaluation design.
17 Oct 2026	T09-T12	All four word-overlap code contributions integrated.
19 Oct 2026	T13-T16	Cross-testing and full lexical evaluation complete; actual results recorded.
21 Oct 2026	T17-T20	All four entailment code contributions integrated.
23 Oct 2026	T21-T24	Entailment tests, pruning comparison and full run complete.
25 Oct 2026	T25-T28	Final configuration, both complete reruns and actual error selection.
27 Oct 2026	T29-T32	Up to 20 errors examined; categories/counts reconciled; detailed examples drafted.
28 Oct 2026	T33-T36	Report contributions, PDF integration and consistency review complete.
29 Oct 2026	T37-T39 checkpoint	First full reproduction, package audit and whole-project mock viva.
30 Oct 2026	T37-T40 final	Freeze verified files; finish checks; verify upload or record submission pending.
31 Oct 2026	Contingency only	Repair essential issues and rerun affected checks; no routine November work.

Official deadline: 11 November 2026, 23:59. The October dates above are the team's internal deadlines.

Shared completion rule: prerequisites complete + evidence + accepted peer review + completion date. Each member contributes to both classifiers, testing, error analysis and six viva topic checks.

Detailed execution and progress guide

This replaces the earlier three-member plan. Use A, B, C and D as stable IDs and record your names in the shared tracker. The plan starts 13 October 2026, completes on 30 October and reserves 31 October for essential repairs. The official deadline remains 11 November 2026, 23:59.

1. Open the shared tracker first

Use `four_member_project_tracker.xlsx` as the single source of task status. It contains Tasks, Daily plan, Experiments, Errors, Contributions and Viva. Put one copy in your team's shared OneDrive/SharePoint folder and open it through Excel for the web if your group uses those services. Coauthor the same file; avoid separately edited copies. The delivered file is local and has not been uploaded or shared automatically.

1. Replace the name placeholders beside A-D on Tasks. Keep A-D unchanged in ownership/reviewer fields so formulas and filters stay consistent.
2. Set the editable As-of date to your check-in date. Alerts use that date, not a hidden assumption about the computer's clock.
3. Filter the task table by Owner to see your work, or Reviewer to see your review queue. Use your own Sheet View during coauthoring if available so filters do not disturb others.
4. Update Status, Evidence link, Review outcome, Finished date and Next action/blocker. Amber cells indicate progress inputs. Do not overwrite formula columns.
5. Only `Done` + nonempty evidence + `Accepted` peer review + a numeric completed date counts as `Complete`. Prerequisite tasks must also be complete; other unfinished tasks remain visible.
6. Update the relevant experiment/error/contribution records; run the daily teach-back. Task completion is not inferred from hours worked.

Initial task states are Not started. All evidence/results are blank. This is a plan, not a claim that coursework has been completed. Owner task counts are balanced at ten each; estimates are planning placeholders and should be revised if real effort differs.

2. Confirm the brief requirements

- Python 3.10 and package versions in requirements.txt.
- Implement first two assessed parts in factcheck.py; leave factchecking_main.py unchanged.
- Word overlap: 40%; lexical scoring/preprocessing/tuning; stated full-credit accuracy target at least 75%.
- Entailment: 40%; provided DeBERTa-v3 model, verified entailment/neutral/contradiction mapping to S/NS; stated target at least 83%; reliable complete execution without out-of-memory errors. When using probability thresholding, take maximum entailment score across examined evidence as specified.
- Error analysis: 20%; up to 10 false positives and 10 false negatives, or all available if fewer; 2-4 meaningful categories collectively; definitions, aggregate counts and three detailed examples with every required field.
- Submit the PDF written report and factcheck.py separately. No word limit is specified.
- Both required commands must work. Accuracy targets alone do not guarantee a grade: the rubric also considers decisions, experiments, efficiency and analysis.
- The brief specifies GenAI Category 3: Editing. Apply the module's permitted-use rules to assessed work; do not invent code results, errors or contributions.

Source: CW.pdf, pages 1-6 and marking grids on pages 8-10. Organisational practices below are team recommendations, not extra required submission files.

3. Balanced responsibilities

Member	Word-overlap code	Entailment code	Independent testing	Analysis/report	Coordination
A	Main lexical score	Overlap pruning	B's inference; B/D lexical components	3 FP + 2 FN; detailed example 1; lexical results	Workflow, integration and final code freeze
B	Tokenization/preprocessing	Model inference, label mapping and S/NS rule	C's aggregation; C/D evidence/memory work	2 FP + 3 FN; detailed example 2; entailment results	Environment and independent reproduction
C	Passage-score aggregation	Sentence segmentation/evidence cleaning	A's score and pruning	3 FP + 2 FN; detailed example 3; analysis section	Data/IR inspection and package audit
D	Threshold evaluation tooling	Bounded processing and evidence-score aggregation	Complete-run metrics/efficiency; A's error records	2 FP + 3 FN; category counts; all-example review and PDF integration	Shared tracker, experiments and verified submission

Every person participates from day one. Leads are responsible for coordination, not isolated ownership. Each reviews another member and teaches back components they did not write.

Participation rules for all four members

- Planning: inspect requirements, confirm availability, trace data flow and contribute tasks.
- Design: propose their component, review all interfaces and jointly approve both pipelines.
- Development: make an identifiable programming contribution to both classifiers, with code evidence.
- Testing: independently check another person's component and participate in full-run diagnosis.
- Analysis: examine real FP/FN examples, review someone else's records and agree categories.
- Writing: contribute original text/results or counts; peer-review before integration.
- Viva: explain every major stage, actual settings/results and the report examples.

4. Daily routine and realistic workload

Reserve about 2-3 focused hours per person daily as a provisional availability assumption, plus unattended model execution. The 3-hour estimates in the tracker describe task effort, not guaranteed daily workload or model runtime. Replace estimates on 13 October after agreeing availability; daily reviews, integration and viva practice are additional work. Measure a representative inference run early. A GPU is optional and must not be assumed.

Start: 10-minute check-in with task IDs, evidence from yesterday and blockers. End: 15-minute review, status update, working commit and rotating teach-back. Record long runs with command, configuration, device, start time and final completion/failure. Never call a partial sample a passed full run.

Statuses: Not started, In progress, Ready for review, Done, Blocked. Review outcome: Pending, Accepted, Changes requested. A peer must inspect evidence and independently verify meaningful behaviour before Accepted. If a material change invalidates evidence, reopen affected tasks and clear acceptance.

5. Detailed tasks and completion criteria

Each task has one accountable owner and a different reviewer. Task dependencies are prerequisites, not suggestions; starting preparatory work is fine, but do not close a task before prerequisites are satisfied.

ID	Section / phase	Owner / reviewer	Dates in October	Dependencies	Work and acceptance criterion
T01	Shared / Planning	A / B	13-14	None	**Agree scope, workflow and contribution rules**. All four agree roles, availability, daily routine and required deliverables.
T02	Shared / Planning	B / C	13-14	None	**Verify Python, packages and model access**. Python 3.10 and required versions recorded; supplied interfaces and loading checked.
T03	Shared / Planning	C / D	13-14	None	**Inspect data, labels and evidence**. Trace real examples; verify IR treatment and evaluation denominator in driver.
T04	Shared / Planning	D / A	13-14	None	**Set up shared tracker and evidence storage**. One shared workbook; stable member IDs; each person traces the whole pipeline.
T05	Shared / Design	A / B	15-15	T01,T02,T03	**Design lexical score and pruning approach**. Record scoring/selection alternatives; all four review names, numbers and negation.
T06	Shared / Design	B / C	15-15	T02,T03	**Design preprocessing and entailment decision**. Record preprocessing options and actual class-mapping verification plan.
T07	Shared / Design	C / D	15-15	T02,T03	**Design aggregation and segmentation**. Agree multiple-passage aggregation and sentence/context handling with all four.
T08	Shared / Design	D / A	15-15	T01,T03,T04	**Design evaluation and bounded execution**. Agree threshold evaluation, logs, batch limits and meaningful checks.
T09	Word overlap / Development	A / B	16-17	T05	**Implement lexical scoring**. Correct score on manually inspected examples; compatible with supplied interfaces.
T10	Word overlap / Development	B / C	16-17	T06	**Implement lexical preprocessing**. Configurable, repeatable preprocessing; inspect entities/numbers/negation.
T11	Word overlap / Development	C / D	16-17	T07	**Implement passage-score aggregation**. Combine passage scores as agreed; cover empty/unhelpful evidence.

ID	Section / phase	Owner / reviewer	Dates in October	Dependencies	Work and acceptance criterion
T12	Word overlap / Development	D / A	16-17	T08	**Implement threshold evaluation tooling**. Reproduce development-set sweep; gold labels used only for evaluation/tuning.
T13	Word overlap / Testing	A / B	18-19	T10,T12	**Test preprocessing and threshold evaluator**. Independently check transformations and below/equal/above threshold behaviour.
T14	Word overlap / Testing	B / C	18-19	T11	**Test lexical evidence aggregation**. Trace multiple/empty passages and inspect final binary decision.
T15	Word overlap / Testing	C / D	18-19	T09	**Test lexical score calculations**. Independently calculate representative scores and record fixes.
T16	Word overlap / Testing	D / A	18-19	T09,T10,T11,T12	**Run and compare full lexical configurations**. Required full command succeeds; actual settings/accuracy/runtime logged; 75% target assessed.
T17	Entailment / Development	A / B	20-21	T16,T05	**Implement overlap pruning for inference**. Pruning compatible with final lexical method; inspect removed supporting evidence.
T18	Entailment / Development	B / C	20-21	T06,T02	**Implement inference class mapping and S/NS rule**. Verify actual label order, premise/hypothesis order and chosen decision strategy.
T19	Entailment / Development	C / D	20-21	T07	**Implement evidence sentence segmentation**. Inspect boundaries/context/empty evidence and long-input handling.
T20	Entailment / Development	D / A	20-21	T08,T18,T19	**Implement bounded model processing and score aggregation**. Control batch/memory use within starter APIs; threshold strategy uses maximum entailment score.
T21	Entailment / Testing	A / B	22-23	T18	**Independently test inference mapping and decisions**. Trace representative model calls and discrete/probability interpretation.
T22	Entailment / Testing	B / C	22-23	T19,T20	**Test segmentation and bounded processing**. Inspect evidence retained and long/empty inputs; no avoidable memory failure.
T23	Entailment / Testing	C / D	22-23	T17	**Compare pruning enabled and disabled**. Comparable settings; record runtime/accuracy and lost-evidence examples.
T24	Entailment / Testing	D / A	22-23	T17,T18,T19,T20	**Run full entailment candidate configuration**. Complete required command; actual accuracy/runtime/device logged; 83% target assessed.

ID	Section / phase	Owner / reviewer	Dates in October	Dependencies	Work and acceptance criterion
T25	Shared / Final evaluation	A / B	24-25	T16,T24	**Integrate fixes and choose final configuration**. Material defects closed; justified final settings recorded; no label hard-coding.
T26	Shared / Final evaluation	B / C	24-25	T25	**Reproduce final lexical results**. Full word_overlap run on frozen candidate; command/version/denominator recorded.
T27	Shared / Final evaluation	C / D	24-25	T26	**Reproduce final entailment results**. Full entailment run on same candidate; predictions captured without changing driver.
T28	Shared / Final evaluation	D / A	24-25	T26,T27	**Select final errors and reconcile metrics**. Select up to 10 FP/10 FN from T27; allocate 5 per member when available; stable IDs.
T29	Error analysis / Analysis	A / B	26-27	T28	**Analyse 3 FP + 2 FN; review B error records**. Five actual records; peer-review B; jointly define 2-4 categories; draft detailed example 1.
T30	Error analysis / Analysis	B / C	26-27	T28	**Analyse 2 FP + 3 FN; review C error records**. Five actual records; peer-review C; jointly define categories; draft detailed example 2.
T31	Error analysis / Analysis	C / D	26-27	T28	**Analyse 3 FP + 2 FN; review D error records**. Five actual records; peer-review D; jointly define categories; draft detailed example 3.
T32	Error analysis / Analysis	D / A	26-27	T28	**Analyse 2 FP + 3 FN; review A and consolidate counts**. Five actual records; peer-review A; reconcile category counts and review all three detailed examples.
T33	Report / Writing and review	A / B	27-28	T26,T29	**Write lexical methods/results and example 1**. Actual experimental evidence; explain decisions and limitations; B reviews.
T34	Report / Writing and review	B / C	27-28	T27,T30	**Write entailment methods/results and example 2**. Mapping/pruning/efficiency explained using real results; C reviews.
T35	Report / Writing and review	C / D	27-28	T29,T30,T31,T32	**Write analysis section and example 3**. Definitions/counts/three required examples reconciled; D reviews.
T36	Report / Writing and review	D / A	27-28	T32,T33,T34,T35	**Integrate PDF and audit report against final code**. Inspect every PDF page; all four approve consistency; no invented results.

ID	Section / phase	Owner / reviewer	Dates in October	Dependencies	Work and acceptance criterion
T37	Shared / Final verification	A / B	29-30	T36	**Coordinate whole-project viva and code freeze**. Each member passes all six topic drills; gaps repaired; final version recorded.
T38	Shared / Final verification	B / C	29-30	T25,T36	**Independently reproduce both required commands**. Specified environment; no missing helpers; unchanged driver; successful complete runs.
T39	Shared / Final verification	C / D	29-30	T38	**Verify evidence, report and final package**. Reconcile final metrics/errors/report; confirm exactly PDF and factcheck.py as separate files.
T40	Shared / Final verification	D / A	29-30	T37,T38,T39	**Verify submission or record ready-to-submit state**. Upload and verify receipt if open; otherwise freeze package and record upload pending.

6. Daily schedule: all four engaged

October date	A	B	C	D	Exit condition
13	T01 scope/availability	T02 environment/model	T03 data inspection	T04 shared tracker	All four begin; identify hardware/access blockers.
14	Review environment; trace driver	Finish T02; trace data	Finish T03; verify IR	Finish T04; organise evidence	Each person explains one end-to-end example.
15	T05 scoring/pruning design	T06 preprocessing/decision design	T07 aggregation/segmentation design	T08 evaluation/memory design	All four approve interfaces and design log.
16	T09 scoring implementation	T10 preprocessing implementation	T11 aggregation implementation	T12 threshold tooling	First small integrated lexical run.
17	Finish T09; explain code	Finish T10; explain code	Finish T11; explain code	Finish T12; explain code	All four lexical code contributions reviewed.
18	T13 test B/D components	T14 test C aggregation	T15 test A scoring	T16 threshold/preprocessing experiments	Meaningful checks and candidate configuration recorded.
19	Fix agreed lexical defects	Review aggregation fixes	Review score fixes	Finish T16 full lexical run	Gate 1: lexical mode works; actual metrics logged.
20	T17 pruning implementation	T18 inference/mapping	T19 segmentation	T20 bounded processing	Small entailment run; each contributes code.
21	Finish T17 and explain	Finish T18 and explain	Finish T19 and explain	Finish T20 and explain	Inference, evidence and memory controls integrated.
22	T21 independently test mapping	T22 test C/D components	T23 pruning comparison	T24 complete-model evaluation	Pruning and decision alternatives measured.
23	Finish T21; inspect errors	Finish T22; resolve failures	Finish T23; review tradeoff	Finish T24 full entailment run	Gate 2: both modes run without memory errors.
24	T25 fixes/final settings	Prepare T26 reproduction	Prepare T27 prediction capture	Prepare T28 selection/metrics	No unresolved core defect; stop optional broad search.

October date	A	B	C	D	Exit condition
25	Finish T25; review settings	Finish T26 full lexical rerun	Finish T27 full entailment rerun	Finish T28 select actual errors	Gate 3: final-run evidence available for analysis.
26	T29: 3 FP/2 FN; review B	T30: 2 FP/3 FN; review C	T31: 3 FP/2 FN; review D	T32: 2 FP/3 FN; review A	All available selected errors examined.
27	Finish T29; T33 example 1	Finish T30; T34 example 2	Finish T31; T35 example 3	Finish T32; reconcile counts	Gate 4: agreed categories/counts and complete draft.
28	Finish T33; review B text	Finish T34; review C text	Finish T35; review D integration	Finish T36 PDF and consistency	Every PDF page inspected; all four sign off.
29	T37 lead rotating mock viva	T38 independently run commands	T39 package audit; viva	Support reproduction; viva	Gate 5: each person explains the whole system.
30	Finish T37; code freeze	Finish T38; record run evidence	Finish T39 final package	T40 upload/verify or pending state	Coursework finished; actual submission state recorded.
31	Coordinate essential repairs only	Rerun affected checks	Refresh analysis/report if needed	Reverify package/upload	Buffer closed; no routine November work.

7. Step-by-step delivery workflow

Step 1: verify setup and understand the starter project

All four read the brief and open the provided framework together. B records environment and actual class interfaces; C verifies how IR is handled and what accuracy counts; A traces the driver and agrees code boundaries; D creates evidence storage and tracking. Preserve the supplied driver and original files. Each member explains one real fact, its passages, label, checker call and evaluation outcome. Verify actual model class order; do not assume indices, APIs or GPU availability.

Step 2: design before separate coding

A proposes score/pruning, B preprocessing/decision, C passage aggregation/segmentation, D evaluation/bounded execution. Record alternatives, rationale and known limitations in the decision log. All four review treatment of names, numbers, negation, empty evidence and context. Use actual starter interfaces when agreeing function ownership. Keep one final reproducible configuration; prediction must never look up a gold label or hard-code outcomes.

Step 3: implement and evaluate word overlap

Integrate A's scoring, B's preprocessing, C's passage aggregation and D's threshold tooling. Start with one functional baseline. Test calculations manually and examine boundary cases. Compare a small number of justified preprocessing/threshold choices, changing one important factor at a time when practical. Tuning on the provided development set is allowed, but reporting on the same set does not establish independent generalisation. D logs actual complete-run metrics; A/B/C independently inspect them. Address below-target performance honestly using observed failures.

Step 4: implement and evaluate entailment

B confirms evidence as premise and fact as hypothesis, verifies label order and maps model outputs to S/NS. C splits/cleans evidence and checks whether context or supporting facts are lost. D controls processing size, checks tokenizer limits/truncation and aggregates evidence scores consistently with the chosen rule. A adds measured low-overlap pruning. Compare pruning enabled/disabled and agreed decision alternatives, recording accuracy and runtime. Cross-test mapped labels, empty/long evidence and complete execution before accepting the 83% target result.

Step 5: freeze final results before error selection

At 24-25 October, close core defects, select final settings and complete both required commands on that exact candidate. Retain code version, command, data/denominator, configuration, device and output. Capture predictions through compatible analysis tooling without modifying the driver. If configuration changes later, rerun affected results and refresh analysis/report; old errors must not be presented as final-model errors.

Step 6: examine and cross-review errors

FP means predicted S, actual NS; FN means predicted NS, actual S. Select up to 10 of each from the final model using a recorded selection method. Assign A/C three FP and two FN each; B/D two FP and three FN each. This yields ten FP, ten FN and five examinations each. If fewer exist, redistribute actual errors as evenly as possible, update allocation and document real totals. Never fabricate errors to fill the template.

Review cycle: A reviews B, B reviews C, C reviews D, D reviews A. All four jointly agree 2-4 categories derived from actual patterns. Use one primary category per examined error for reconciled totals; secondary observations can remain in notes. D consolidates counts and reviews all three detailed examples. A/B/C write one detailed example each, with fact text, actual label, predicted label, category and 1-3 sentences of explanation. D's substantive analysis contribution is five examined errors plus definitions/count reconciliation and evidence review; the brief requires three detailed examples, not four.

Step 7: write, reproduce and freeze

A writes lexical choices/results; B entailment choices/results; C analysis; D integrates and checks the PDF. All four contribute limitations and approve claims. Inspect every PDF page. B independently reproduces both commands with required versions; C reconciles package/results/report; A freezes code; D verifies upload receipt or explicitly records pending submission if the dropbox is not open. Recommended experiment logs and task evidence support your work; submit the required PDF and code separately.

8. Evidence records and progress rules

Experiments sheet: exact command, code/configuration, mode, data/denominator, device, threshold, pruning, numeric accuracy and runtime, conclusion and reviewer. Empty results stay empty until measured. Its summary only includes completed runs.

Errors sheet: fixed E01-E20 placeholders with allocation; actual fact/evidence/labels/category/explanation remain blank until inspected. Filled placeholders are not completed analysis. Record real FP/FN, category and accepted cross-review; reconcile actual totals against available errors. The Reviewed count requires complete fact/evidence/category/explanation fields, accepted review and labels consistent with FP/FN. Enter 2-4 distinct category names and definitions in the category table to calculate counts; the unlisted-category count should be zero. For fewer errors, mark unused rows Not available and explain, rather than inventing text.

Contributions sheet: date, member, task ID, phase, concrete contribution, evidence, peer and learning outcome. Record reviews and knowledge sharing as well as code. Add rows inside the table so filters include them. The Tasks summary is bounded to the 40 planned tasks: any extra tasks require extending summary formulas and gate formulas, not just appending an uncounted row.

Viva sheet: six topic checks for each member. Passed requires a peer's accepted evidence-based demonstration. Overall project task completion does not mean every member has passed the viva checks.

Record design decisions and defects in the shared evidence folder using these templates and put links in Tasks:

```
Decision ID/date | Question | Options | Evidence/run IDs | Choice | Rationale/limitation
  Agreed by
Defect ID | Observed behaviour | Reproduction | Impact | Owner | Fix/version
  Peer verification | Status
Daily closeout | Member | Completed task/evidence | Next action | Blocker
  Runs still executing | Teach-back gap
```

9. Viva: everybody learns the whole project

Six topics per member: purpose/data/IR; word overlap; entailment/model mapping; evidence/pruning/memory; experiments/evaluation limitations; error analysis/report. Use the Viva sheet to track each of the 24 topic demonstrations. Begin at each milestone, not only 29 October.

Rotate four roles: examiner, candidate, code/evidence checker, observer. Each candidate must trace code and answer from actual results. Repeat failed topics after a walkthrough. Suggested questions:

1. Why can high word overlap fail to prove a fact?
2. How does your exact score and passage aggregation work?
3. What preprocessing choices mattered, and what did experiments show?
4. How was the threshold selected and how do boundary decisions work?
5. Which input is premise/hypothesis, and how did you verify model labels?
6. How do three classes become S/NS in your implementation?
7. Why use maximum evidence scores when thresholding?
8. What evidence can segmentation or pruning lose?
9. How do batching and truncation affect accuracy/memory?
10. What are the actual final accuracies and denominators? How is IR handled?
11. Why is development-set tuning/reporting a limitation?
12. Explain one real FP and FN and why your category fits.

13. Which change helped most, according to recorded evidence?
14. Trace one example from loading to metric calculation.
15. What did you personally design, code, independently test and analyse?
16. How can another person reproduce the result?

Answer pattern: decision -> reason -> actual evidence -> limitation. Know all three report examples, not just the one you wrote.

10. Milestones, recovery and final checks

Gate	Date	Pass condition	Recovery priority
1	19 October	Complete lexical run and recorded metrics	Fix core path before extra comparisons
2	23 October	Both modes complete reliably	Check mapping/evidence/memory before sophistication
3	25 October	Final settings/results/predictions reproducible	Stop optional broad tuning; finish real error selection
4	27 October	All available selected errors, categories/counts and draft	Redistribute missing records; cut decorative report work
5	29 October	Reproduction and each member's six-topic practice	Repair required gaps on 30-31 October

No GPU: benchmark CPU early and use measured bounded processing/pruning. Model access failure: B diagnoses while others continue independent lexical/data work, escalating unresolved access to course support. Member absence: reassign explicit task cards and teach them missed components on return. Merge conflicts: agree block ownership, make small commits and integrate daily. Accuracy below target: fix correctness first and conduct limited evidence-based tuning; report honestly. Late changes: reopen acceptance and regenerate dependent evidence.

Critical path: setup/data -> integrated classifiers -> final full runs -> actual error selection -> report -> independent verification -> freeze/submission. Reduce optional metrics, broad searches and decorative report work before sacrificing required analysis, full verification or shared understanding.

Run in the prescribed environment:

```
python factchecking_main.py --mode word_overlap
python factchecking_main.py --mode entailment
```

The optional --cuda flag requires a suitable enabled GPU/CUDA installation. Do not assume it is necessary for the two required commands.

[ ] Both complete commands succeed; final actual metrics retained.

[ ] Python/package versions match; driver unchanged; no missing unsubmitted helper dependencies.

[ ] Report matches the final code/configuration/predictions.

[ ] Required available FP/FN, 2-4 categories, counts and three detailed examples included.

[ ] Every PDF page inspected; all four approve.

[ ] Each member has code contributions in both classifiers, analysis and independent tests.

[ ] All 24 member/topic viva checks accepted after genuine demonstration.

[ ] PDF and factcheck.py separately uploaded with receipt checked, or clearly marked ready with upload pending.

[ ] Coursework complete by 30 October; necessary repair buffer closes 31 October.

Task status, completed deliverables, individual understanding and actual submission are separate checks. Keep each honest and visible.